

Taller Microservicios

1. Experimento

Título del experimento	Implementación de microservicios en el sistema de control térmico
Propósito del experimento	Comprobar si el uso de microservicios con patrones de diseño beneficia la facilidad de modificación de un sistema sin afectar la disponibilidad.
Resultados esperados	Tiempo de desarrollo del servicio Places.
Infraestructura computacional requerida	- Instancias de MVs en GCP. - Cloud functions en GCP. - Script IaC para despliegue. - Computador personal para acceder a los servicios web.
ASRs involucrados	Yo como arquitecto del sistema de control térmico, dado que el sistema se encuentra desplegado, cuando se presente el requerimiento de integrar un nuevo servicio, quiero que el equipo de desarrollo lo despliegue y esto no debe tomar más de 2 horas.
Descripción del experimento	En este experimento la aplicación Monitoring App se encuentra dividida en dos servicios inicialmente: variables y medidas. Cada servicio se desplegará en máquinas individuales donde un servicio está dockerizado y el otro no. Para unificar la aplicación, se configura un API Gateway basado en Kong DB-less. Luego, se creará una función <i>serverless</i> que consumirá un API externa con una frecuencia definida para extraer datos de temperatura de forma masiva y persistir los registros en una DB usando el servicio Measurements. Finalmente, usted debe crear una nueva función <i>serverless</i> que importe datos de oxígeno de un API externa. El microservicio de Measurements valida que el lugar de cada medición exista. Esta validación debe hacerla invocando el microservicio Places, cuyo código es provisto por el equipo docente. Se espera

que usted lo integre en el sistema. Usted debe cronometrar el desarrollo de la nueva función *serverless* y del despliegue del microservicio Places para comprobar si se cumple el ASR.

1.1. Estilos de arquitectura

Estilos de Arquitectura asociados	Análisis (Atributos de calidad que favorece y desfavorece)
Microservicios	- Favorece escalabilidad y facilidad de modificación. - Desfavorece la realización de pruebas de integración.
Serverless*	- Favorece la escalabilidad, disponibilidad y eficiencia de costos. - Desfavorece latencia, seguridad y facilidad de pruebas. *Algunos autores clasifican serverless como un patrón.

1.2. Patrones de diseño

Patrones de diseño	Descripción
API Gateway	El patrón de API Gateway centraliza y gestiona las solicitudes de clientes a servicios backend, simplificando la arquitectura, mejorando la seguridad y el rendimiento, y facilitando la administración y el monitoreo de la API.

Service Registry	Es un registro donde se definen los servicios que el sistema es capaz de responder y permiten al cliente la localización de estos.
Remote Procedure Invocation	La Invocación de Procedimiento Remoto (RPI) permite a un programa solicitar servicios de otro en la red como si fuera una llamada local, facilitando la modularidad y escalabilidad de las aplicaciones.
DB por servicio	El patrón de DB por microservicio asigna una base de datos a cada microservicio, promoviendo la independencia y escalabilidad, aunque puede complicar la consistencia de datos y transacciones.
Descomposición por subdominios	El patrón de descomposición por subdominios en microservicios divide una aplicación en componentes pequeños, cada uno centrado en una parte específica del negocio, lo que facilita la evolución y escalabilidad independiente.
Despliegue por contenedores	El despliegue por contenedores ejecuta aplicaciones en entornos aislados y portátiles, mejorando consistencia y escalabilidad. Ejemplos: Docker y Kubernetes.
Despliegue por máquina virtual	El despliegue por máquinas virtuales permite ejecutar múltiples aplicaciones en un solo servidor físico que puede ser remoto, ofreciendo aislamiento y eficiente uso de recursos.

1.3. Elementos de arquitectura

1.3.1. Modelo de datos

En este taller desplegará dos microservicios que componen la aplicación web de monitoreo que ha desarrollado durante el semestre en el laboratorio. El primer microservicio gestiona la creación y consulta de las medidas de monitoreo “measurements”, y el segundo microservicio gestiona la creación y consulta de las variables del sistema de monitoreo. En la figura 1, puede observar el modelo de datos, donde una variable se compone de cero o muchas medidas “measurement”. En este taller, vamos a tomar estos dos como subdominios del sistema, y serán desplegados en dos microservicios.

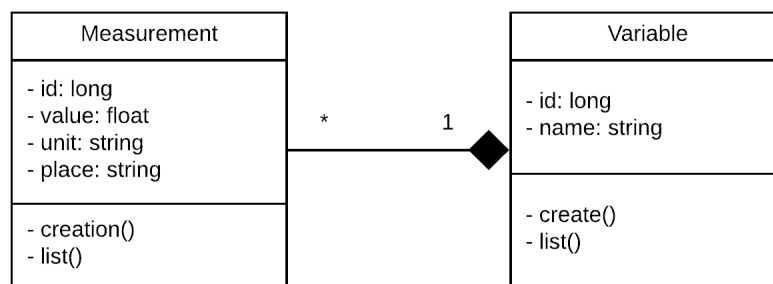


Figura 1. Modelo de datos

1.3.2. Diagrama de despliegue

De acuerdo con el diagrama de despliegue (figura 2), cada microservicio será desplegado en una instancia VM de GCP, además, tendrán bases de datos por separado, con el objetivo de mitigar problemas como “cuellos de botella” por el uso de una única base de datos para toda la aplicación. Utilizará Kong como API Gateway, para direccionar las peticiones a cada microservicio como sea necesario. Adicionalmente, existe comunicación entre los dos microservicios a través del API Gateway Kong, ya que es necesario el intercambio de información entre estos. El microservicio “variables” será desplegado a través de contenedores.

Existe un tercer microservicio desplegado a través de una función Serverless (en la figura referido como `api-consumption`) que se encargará de consumir “Measurements” en lote de un API externa y persistirlas en la base de datos correspondiente. Esta función se activa periódicamente a través de un cron (llamado Scheduler en GCP).

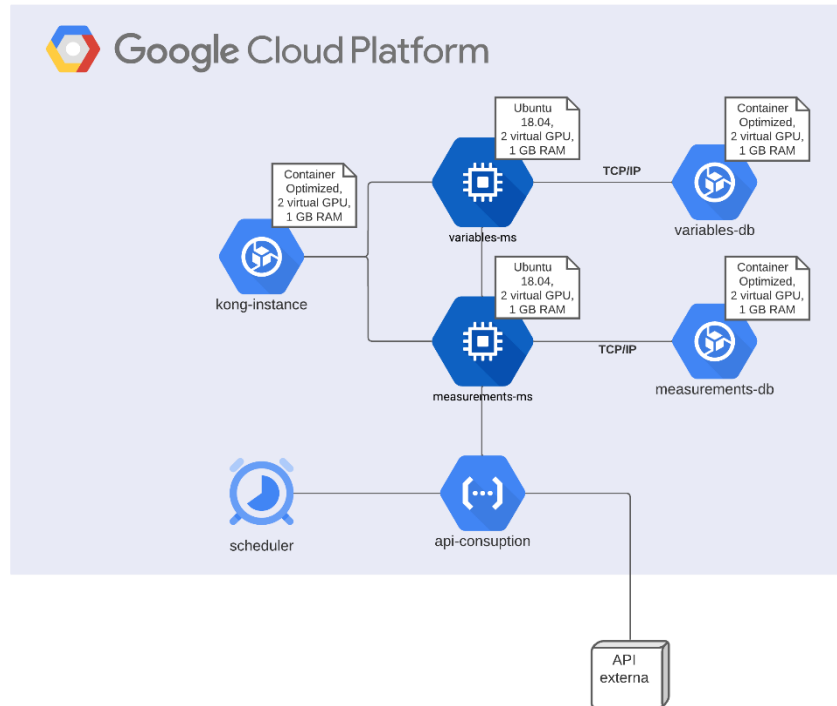


Figura 2. Vista de despliegue

1.4. Tecnologías

Tecnologías asociadas	Selección y Justificación
Frameworks	Django: Es un framework Web robusto y de rápida implementación que usa el modelo MVC.
Lenguajes de programación	Python: Es el lenguaje principal de desarrollo de Django.
Plataforma de despliegue	- GCP: La nube computacional permite escalabilidad y flexibilidad en la infraestructura. GCP es la nube patrocinadora del proyecto. - GCP Deployment Manager: Es un servicio de despliegue de infraestructura (IaC) que automatiza la creación y gestión de recursos en GCP.
Bases de datos	PostgreSQL: Es una base de datos SQL popular que favorece la disponibilidad, la latencia y la flexibilidad.

Herramientas de análisis	• Navegador: Es la herramienta que permite interactuar con los microservicios desplegados (variables y measurements).
Librerías	• Psycopg2: Es la librería que permite la integración entre Django y PostgreSQL.
Otras	• Kong Gateway: Es un API gateway optimizado para arquitecturas distribuidas o con microservicios. • Docker: Es una tecnología de contenerización que permite la distribución y ejecución rápida de aplicaciones.

DOCKER

Docker empaqueta software en unidades estandarizadas llamadas contenedores que incluyen todo lo necesario para que el software se ejecute, incluidas bibliotecas, herramientas de sistema, código y tiempo de ejecución. Con Docker, puede configurar y ajustar el ambiente de ejecución de las aplicaciones rápidamente con la certeza de saber que su código correrá.

Hay dos conceptos clave que es necesario tener claros para utilizar correctamente Docker:

- **Imagen:** Es una especie de plantilla, una captura del estado de un contenedor. Se podría decir que una imagen de un contenedor es como un [snapshot](#) de una máquina virtual, pero mucho más ligero. Por ejemplo, una imagen puede contener un sistema operativo Ubuntu, un servidor Apache y una aplicación web instalada. Las imágenes se utilizan para crear contenedores y son inmutables.
- **Contenedor:** Es una instancia en ejecución de una imagen. Un contenedor es el que ejecuta una aplicación, base de datos, etc. Haciendo una analogía con el concepto de máquina virtual, ejecutar un contenedor a partir de una Imagen, es como si restauráramos una máquina virtual a partir de un snapshot (no es lo mismo, pero se hace esta analogía para comprender mejor el procedimiento). A partir de una única imagen, podemos ejecutar varios contenedores.

Docker ofrece una forma estándar de desplegar su código. Es un sistema operativo para contenedores. De manera similar a cómo una máquina virtual virtualiza (elimina la necesidad de administrar directamente) el hardware del servidor, los contenedores virtualizan el sistema operativo de un servidor como se observa en la fig. 4. Docker se instala en cada máquina y proporciona comandos sencillos que puede utilizar para crear, iniciar o detener contenedores.

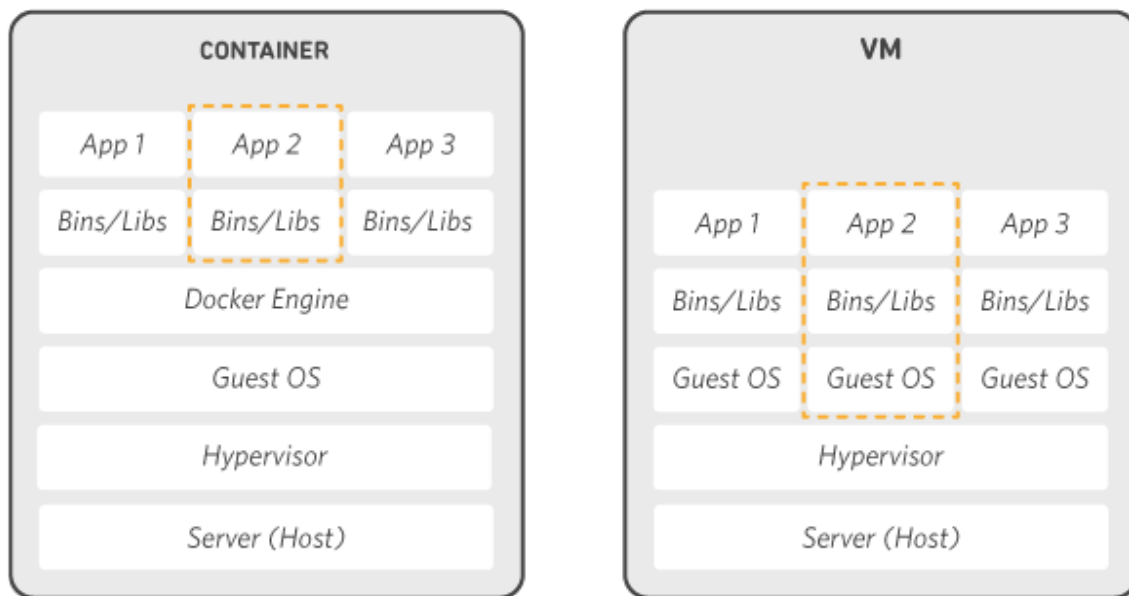


Figura 4. Arquitectura de un contenedor vs máquina virtual

Si desea obtener más información acerca de Docker, puede consultar el siguiente enlace:

<https://www.docker.com/why-docker>

En la tabla 1 puede ver una lista de comandos de Docker para la creación y administración de imágenes y contenedores. Algunos de estos comandos son utilizados en este taller.

Función	Comando
Crear imagen	<code>docker build -t <nombre_imagen> .</code>
Borrar imagen	<code>docker rmi <nombre_imagen></code>
Listar imágenes	<code>docker images</code>
Crear contenedor	<code>docker run -name <nombre_contenedor> <nombre_imagen></code>
Borrar contenedor	<code>docker rm <nombre_contenedor></code>
Listar contenedores	<code>docker ps -a</code>
Iniciar contenedor	<code>docker start <nombre_contenedor></code>
Detener contenedor	<code>Docker stop <nombre_contenedor></code>
Ingresar en la consola de un contenedor	<code>docker exec -i -t <nombre_contenedor> /bin/bash</code>

Tabla 1. Comandos de Docker

En cada comando es posible establecer varios parámetros. Si desea conocer la lista completa de comandos y parámetros de cada comando Docker, puede ingresar al siguiente enlace:

<https://docs.docker.com/engine/reference/commandline/docker/#child-commands>

2. Actividades

Este taller debe realizar de forma **individual**.

Nota: Los comandos de este laboratorio se encuentran en el [README del repositorio](#) para facilitar su uso. Les recomendamos seguir esta guía y cuando encuentre un comando puede dirigirse al README para copiar y pegar donde necesite.

2.1. Desplegar la infraestructura

2.1.1. Ejecutar Script IaC

Para desplegar la infraestructura de este laboratorio se usará un script de IaC. Tome de guía el punto 2.1. "Desplegar la infraestructura usando IaC" del [laboratorio de Circuit Breaker](#) y tenga en cuenta lo siguiente:

- El repositorio de este laboratorio es diferente y se usará la rama master. Repositorio en GitHub: <https://github.com/ISIS2503/ISIS2503-Microservices-AppDjango.git>
- En el punto 2., en CloudShell, ejecute los siguientes comandos para tener en cuenta el nuevo repositorio:

```
>>> git clone https://github.com/ISIS2503/ISIS2503-Microservices-AppDjango.git
>>> cd ISIS2503-Microservices-AppDjango/
```

- En el archivo deployment.yaml de este laboratorio se crean 4 reglas de firewall y 5 instancias: Kong y dos microservicios (backend y DB para cada uno). Revise el archivo. Las máquinas Kong, el backend de Variables y las bases de datos utilizan el sistema operativo Container Optimized OS pues estarán ejecutando contenedores de Docker. Y la máquina del microservicio de Measurements ejecutará el servicio como se han ejecutado Monitoring App en otros laboratorios.
- Los nombres de los recursos de este despliegue empiezan por "msd-" por "Microservices Deployment".
- En el punto 8, utilice un nombre representativo de este laboratorio. Por ejemplo: "ms-deployment".
- **Recuerde esperar 5 minutos** mientras se ejecutan los scripts de inicio.

2.2. Configuración del API Gateway Kong

En este laboratorio se utiliza Kong como API gateway. Este componente será el encargado de recibir las peticiones de un cliente o dispositivo y dirigir las al servicio correspondiente. Inicialmente se configuran dos servicios: Variables y Measurements, con rutas /variables y /measurements respectivamente. Al igual que en el laboratorio de Circuit Breaker, en este taller se usa Kong en modo DB-less y se configura con un archivo declarativo que es `kong.yaml`. Siga las siguientes indicaciones para observar el archivo.

1. Conéctese a la máquina virtual de Kong (`msd-kong-instance`) por SSH.
2. Ubíquese en la carpeta del repositorio. Esta vez se encuentra en la ruta `/home/labs`.

```
>>> cd /home/labs/ISIS2503-Microservices-AppDjango
```

3. Tenga en cuenta que el script `laC` clonó el repositorio e inició un contenedor Kong con configuración `kong.yaml` en el puerto 8000. Los comandos ejecutados por el script los puede observar en la siguiente imagen (líneas 104-113 archivo `deployment.yaml`).

```

102 - key: startup-script
103   value: |
104     #!/bin/bash
105     sudo mkdir /home/labs
106     cd /home/labs
107     sudo git clone https://github.com/ISIS2503/ISIS2503-Microservices-AppDjango.git
108     cd ISIS2503-Microservices-AppDjango
109     docker network create kong-net
110     docker run -d --name kong --network=kong-net --restart=always \
111       -v "$(pwd):/kong/declarative/" -e "KONG_DATABASE=off" \
112       -e "KONG_DECLARATIVE_CONFIG=/kong/declarative/kong.yaml" \
113       -p 8000:8000 kong/kong-gateway

```

4. Entre a visualizar el archivo `kong.yaml` con el editor Vi. Puede salir del editor escribiendo `":q"` y Enter. En el momento que necesite editar el archivo, puede entrar al modo edición oprimiendo `"i"` y cuando termine debe salir del modo de edición con la tecla ESC y escribir `":wq"` y Enter. Puede encontrar más información en [este tutorial](#).

```
>>> sudo vi kong.yaml
```

5. Observe la estructura del archivo. En la configuración existen dos categorías principales: `services` y `upstreams`. En `upstreams` se definen los grupos de servidores que responderán peticiones. En

este caso solo se configura una máquina por upstream. En el lado de servicios se definen el nombre de un servicio, el protocolo a utilizar, la ruta por la que se responden las solicitudes y el upstream que se va a usar para procesar las peticiones.

```
ISIS2503-Microservices-AppDjango > ! kong.yml
1  _format_version: "2.1"
2
3  services:
4    - name: variables_service
5      host: variables_upstream
6      protocol: http
7      routes:
8        - name: variables
9          paths:
10            - /variables
11            strip_path: false
12
13    - name: measurements_service
14      host: measurements_upstream
15      protocol: http
16      routes:
17        - name: measurements
18          paths:
19            - /measurements
20            strip_path: false
21
22  upstreams:
23    - name: variables_upstream
24      targets:
25        - target: 10.128.0.84:8080
26          weight: 100
27
28    - name: measurements_upstream
29      targets:
30        - target: 10.128.0.85:8080
31          weight: 100
32
```

6. Este archivo permite agregar más servicios y servidores respondiendo a peticiones del sistema. Si se modifica, se debe reiniciar el contenedor de Kong para que cargue la nueva configuración. Puede reiniciar Kong con el siguiente comando.

```
>>> docker restart kong
```

2.3. Ejecución del servicio de Variables

Para este taller, usted creará una imagen de Docker del servicio de Variables y la ejecutará dentro de un contenedor. Esto es valioso pues Docker le permite crear programas, servidores o sistemas fácilmente

distribuibles y ejecutables. Con las siguientes indicaciones se espera que usted pueda construir una imagen de Docker básica.

1. Conéctese por SSH a la máquina del servicio de variables (msd-variables-ms).
2. Ingrese al directorio de variables del repositorio con el comando:

```
>>> cd /home/labs/ISIS2503-Microservices-AppDjango/variables
```

3. Ya en el directorio de variables, usted podrá crear la imagen del servicio para Docker. La creación de la imagen es un proceso sencillo pues el repositorio ya incluye el Dockerfile para crear la imagen. Este archivo contiene las instrucciones para construir la imagen como tomar como base otra imagen, configurar variables o ejecutar comandos.
- Visualice el archivo Dockerfile dentro de variables. Podrá ver un contenido como la siguiente imagen.

```

ISIS2503-Microservices-AppDjango > variables > Dockerfile > ...
1  FROM python:3.8.10-alpine3.13
2
3  WORKDIR /app
4
5  # Copy the django app to the working directory
6  COPY . /app
7
8  # Set the environment variables
9  ENV VARIABLES_DB_HOST 10.128.0.82
10
11 # Install Python dependencies
12 RUN apk update && apk add postgresql-dev gcc python3-dev musl-dev
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 ENTRYPOINT ["sh", "init.sh"]
  
```

Contenido de Dockerfile

Dentro del Dockerfile encuentra varios comandos que guían la creación de la imagen. Las descripciones de los comandos usados se encuentran en la siguiente tabla.

Comando	Parámetros	Descripción
WORKDIR	Ruta de directorio. Ej: /app	Se crea el directorio indicado en la imagen y se le indica a Docker que los siguientes comandos se ejecutaran dentro de esta ruta.
COPY	• Ruta de directorio/archivo	Docker copia el archivo o

	origen. Ej: . • Ruta de directorio/archivo destino. Ej: /app	directorio origen desde el host (su computador por ejemplo) y lo copia a la ruta destino dentro de la imagen.
ENV	• Nombre de la variable. Ej: VARIABLES_DB_HOST • Valor de la variable. Ej: 10.128.0.82	Configura las variables de ambiente de la imagen con su nombre y valor. Estas son útiles dentro de los contenedores.
RUN	Comando a ejecutar dentro de WORKDIR. Ej: apk update	Ejecuta el comando especificado dentro de la imagen. Sólo se ejecuta al momento de la creación de la imagen.
ENTRYPOINT	Lista con un comando y los argumentos. Ej: ["sh", "init.sh"]	Se especifica el comando que el contenedor ejecutará una vez creado o iniciado.
FROM	Imagen base. Ej: python:alpine	Se toma como base la imagen indicada para aprovechar su software o configuraciones.

- En síntesis, el Dockerfile creará una imagen basada en python 3.8.10 con el repositorio actual. Luego, configurará las variables de ambiente (IP de la DB de variables) e instalará las dependencias del proyecto. Y, por último, indica qué se debe ejecutar dentro del contenedor cuando inicie.
- En ENTRYPOINT se indica a Docker que se debe ejecutar el archivo init.sh al iniciar el contenedor. Ese archivo contiene los comandos de la siguiente imagen.

```

ISIS2503-Microservices-AppDjango > variables > $ init.sh
1  #!/bin/bash
2
3  python3 manage.py makemigrations
4
5  python3 manage.py migrate
6
7  python3 manage.py runserver 0.0.0.0:8080
  
```

Contenido de init.sh

Los comandos son exactamente los mismos que usted ha usado siempre que quiere correr el proyecto Django de los talleres del curso. Primero se crean las migraciones del proyecto, luego se migran a la base de datos y se finaliza ejecutando el servidor web.

- Proceda a la creación de la imagen usando la terminal SSH de la MV y ejecutando este comando:

```
>>> docker build -t variables-app .
```

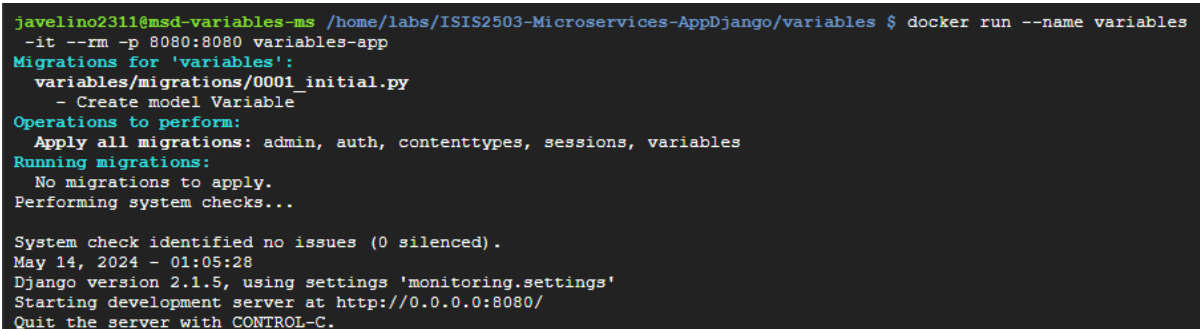
- Cuando finalice el proceso ya tendrá localmente en la máquina la imagen del servicio de variables bajo el nombre de "variables-app". Esto lo usará al correr un contenedor.

4. Finalice corriendo el contenedor Docker que ejecutará el microservicio de measurements.

- Ejecute este comando cambiando antes las direcciones IP **privadas** por las de su despliegue. Tenga en cuenta que el flag `--rm` elimina el contenedor una vez acabe el programa (ej: Ctrl+C). Para volver a correr el servicio de variables debe ejecutarlo de nuevo.

```
>>> docker run --name variables -it --rm -e  
VARIABLES_DB_HOST=<ip-variables-db> -p 8080:8080 variables-app
```

- Podrá ver que la terminal muestra los mismos mensajes que cuando intenta correr Django sin docker.



```
javelino2311@msd-variables-ms /home/labs/ISIS2503-Microservices-AppDjango/variables $ docker run --name variables  
-it --rm -p 8080:8080 variables-app  
Migrations for 'variables':  
  variables/migrations/0001_initial.py  
    - Create model Variable  
Operations to perform:  
  Apply all migrations: admin, auth, contenttypes, sessions, variables  
Running migrations:  
  No migrations to apply.  
Performing system checks...  
  
System check identified no issues (0 silenced).  
May 14, 2024 - 01:05:28  
Django version 2.1.5, using settings 'monitoring.settings'  
Starting development server at http://0.0.0.0:8080/  
Quit the server with CONTROL-C.
```

Mensajes del terminal luego ejecutar docker run.

2.4. Ejecución del servicio de Measurements

El servicio de measurements se encarga del manejo de medidas dentro del sistema del control térmico. Estas medidas corresponden a una variable y a un lugar. Al usar el estilo de microservicios, la administración de medidas se convierte en un servicio independiente. Sin embargo, Measurements se conecta con el servicio de Variables para confirmar que existen la variable con la que se crea una medida o consultar información más a fondo de la variable. Siga las siguientes instrucciones para observar la conexión de estos dos microservicios y la ejecución del servicio de Measurements.

1. Conéctese por SSH a la instancia del servicio de Measurements (msd-measurements-ms).
2. Ingrese al directorio de measurements en el repositorio clonado. Ejecute:

```
>>> cd /home/labs/ISIS2503-Microservices-AppDjango/measurements
```
3. Visualice (usando nano) el archivo monitoring/settings.py donde podrá observar en la última línea la definición de la variable global *PATH_VAR* que tiene la URL del microservicio de variables. Django intenta leer las variables de entorno *VARIABLES_HOST* y *VARIABLES_PORT* para definir la ip y el puerto del microservicio, si no se configuran en el sistema operativo toma los datos por defecto (10.128.0.84 y 8080).

```
ISIS2503-Microservices-AppDjango > measurements > monitoring > settings.py
130
131 # Extra places for collectstatic to find static files.
132 STATICFILES_DIRS = (
133     os.path.join(PROJECT_ROOT, 'static'),
134 )
135
136 PATH_VAR = "http://" + os.environ.get("VARIABLES_HOST", "10.128.0.84") + ":" + os.environ.get("VARIABLES_PORT", "8080") + "/variables"
```

Por otra parte, visualice el archivo measurements/views.py donde se define la función *check_variable* que usa la variable global definida y envía una petición GET al microservicio de variables para verificar que la variable indicada exista en el sistema.

```
ISIS2503-Microservices-AppDjango > measurements > measurements > views.py
10
11 def check_variable(data):
12     r = requests.get(settings.PATH_VAR, headers={"Accept": "application/json"})
13     variables = r.json()
14     for variable in variables:
15         if data["variable"] == variable["id"]:
16             return True
17     return False
18
```

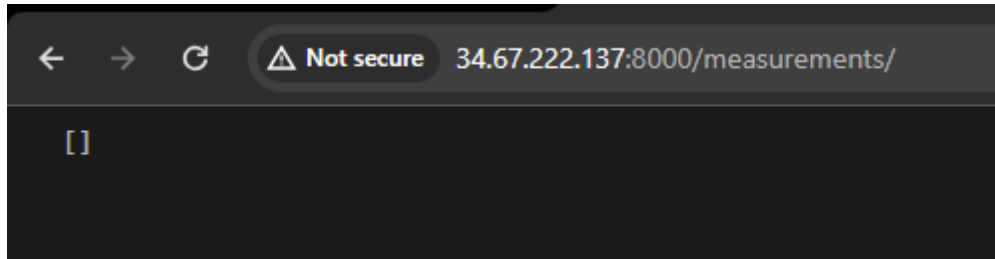
4. Ejecute el servicio de measurements.
 - Cree las migraciones de la aplicación.

```
>>> sudo python3 manage.py makemigrations
```
 - Aplique las migraciones en la base de datos.

```
>>> sudo python3 manage.py migrate
```
 - Corra el servidor Django en el puerto 8080.

```
>>> sudo python3 manage.py runserver 0.0.0.0:8080
```

5. Asegúrese de que los servicios variables y measurements estén funcionando. Acceda desde un navegador a <http://<ip-externa-kong>:8000/variables> y <http://<ip-externa-kong>:8000/measurements>. Debe ver en ambas URLs una página como la siguiente.

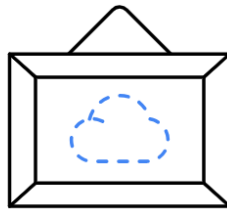
**NOTA:**

- Compare el despliegue de los microservicios variables y measurements. ¿Qué pros y contras encuentra en ambas formas de despliegue?

2.5. Crear la función serverless – *api-consumption*

En esta parte, crearemos una función (serverless) que se encarga de consultar un API externa que brinda datos de medidas (por ej., temperatura) e invoca al microservicio “measurements” para que este a su vez los almacene en la base de datos correspondiente. Vale la pena mencionar que esta función no es alcanzable desde el APIGateway ya que la función será llamada periódicamente por un “Scheduler” que especificaremos más adelante.

- Busque el servicio *Cloud Run* en el panel de búsqueda superior o en el menú lateral izquierdo en el submenú Sin Servidores. Presione el botón “Crear Servicio” que se muestra en la captura. Paso seguido le pedirá habilitar varias APIs de GCP, habilítelas.



Ejecuta tu aplicación en una plataforma completamente administrada

Para comenzar, crea un servicio. Cada servicio de Cloud Run tiene un extremo único y ajusta de forma automática la escala de tu contenedor, código o función. [Más información](#)

Si haces clic en "Crear servicio", se habilitará la API de Cloud Run Admin.

[Crear servicio](#)

[Usar el inicio rápido](#)

- Complete el formulario de creación con la siguiente información (ver captura que sigue):
 - **Tipo de servicio:** Función
 - **Nombre:** api-consumption
 - **Región:** us-central1
 - **Runtime/Ejecución:** Python 3.9
 - **Autenticación:** Usar Cloud IAM. Permitir invocaciones sin autenticar.
 - **Contenedores:**
 - **Configuración:**
 - **Memoria:** 256 MB
 - **Variables y secretos:**
 - API_PATH:
<https://raw.githubusercontent.com/ISIS2503/ISIS2503-Microservices-AppDjango/master/data/temperatura.json>
 - MS_PATH:
http://<IP_PUBLICA_MEASUREMENTS_MS>:8080/createmeasurements/
 - **Tiempo de espera:** 120 segundos (si no se completa la función en este tiempo, se finalizará la función)
 - **Escalamiento:**
 - **Número mínimo de instancias:** 0
 - **Número máximo de instancias:** 2
 - **Configuración del entorno de ejecución, la compilación, las conexiones y la seguridad:**
 - **Memoria asignada:** 256 MB
 - **Tiempo de espera:** 120 segundos (si no se completa la función en este tiempo, se finalizará la función)
 - **Número mínimo de instancias:** 0
 - **Número máximo de instancia:** 2

- **Variables de entorno** - estas variables serán usadas en el código de la función para saber: i) cuál es el API externa desde donde se consumirán las mediciones; y ii) dónde está desplegado el microservicio measurements:
 - API_PATH: <https://raw.githubusercontent.com/ISIS2503/ISIS2503-Microservices-AppDjango/master/data/temperatura.json>
 - MS_PATH: http://<IP_PUBLICA_MEASUREMENTS_MS>:8080/createmeasurements/
- Habilite el api de Cloud Build si la interfaz lo pide.
(note que aquí debe reemplazar **IP_PUBLICA_MEASUREMENTS_MS** por la ip correspondiente en su despliegue)

Cloud Run

← Crear servicio

Cada servicio expone un extremo único y ajusta automáticamente la escala de la infraestructura subyacente para controlar las solicitudes entrantes. No se puede cambiar el nombre del servicio ni la región más adelante.

Artifact Registry Docker Hub

☐ Implementar una revisión desde una imagen de contenedor

GitHub

☐ Implementar continuamente a partir de un repositorio (de origen o de función)

Función

☒ Usar un editor directo para crear una función

Configurar

Nombre de Servicio *

api-consumption

Región *

us-central1 (Iowa)

[¿Cómo se selecciona la región? ↗](#)

URL del extremo ⓘ

https://api-consumption-191049809556.us-central1.run.app

Tiempo de ejecución *

Python 3.9

Autenticación *

☒ Usar Cloud IAM para autenticar las solicitudes entrantes
Cloud IAM autorizará todas las invocaciones del extremo de este servicio.

☒ Permitir invocaciones sin autenticar
Configura una política de Cloud IAM que permite el acceso sin una credencial.

☐ Autenticación obligatoria
Administrar los usuarios autorizados con Cloud IAM.

Contenedores, volúmenes, Herramientas de redes y seguridad

Contenedores Volúmenes Redes Seguridad

Editar contenedor

URL de la imagen del contenedor
gcr.io/cloudrun/placeholder

Puerto de contenedor
8080

Las solicitudes se enviarán al contenedor de este puerto. Recomendamos detectar en \$PORT, en lugar de en este número específico.

Configuración Variables y secretos Activaciones de volúmenes

Variables del entorno

Nombre	Valor
API_PATH	https://raw.githubusercontent.com/ISIS2503/ISIS
MS_PATH	http://<IP_PUBLICA_MEASUREMENTS_MS>:8080

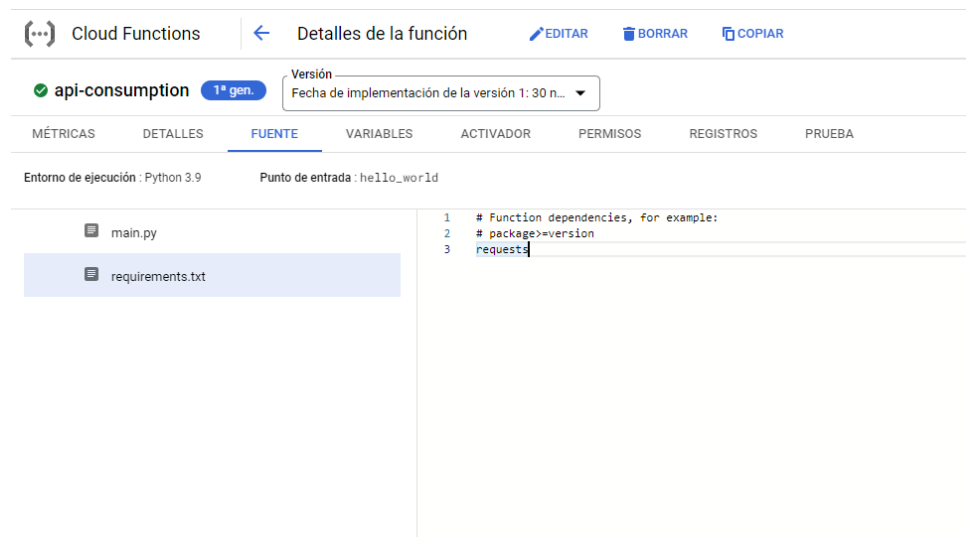
+ Agregar variable

- Presione “Continuar o siguiente” para proseguir con los pasos de creación para la definición del código de la función. Seleccione como entorno de ejecución Python 3.9 (note que hay diferentes lenguajes) y copie el siguiente código en el archivo main.py (asegúrese que al copiar & pegar el código en el editor de la función, el código queda con la indentación apropiada):

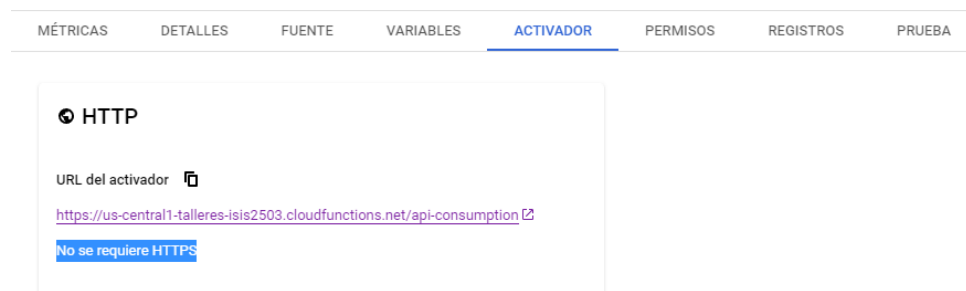
```
import requests
import json
import os

def hello_http(request):
    data = requests.get(os.environ.get('API_PATH'), headers={"Accept": "application/json"})
    json_data = data.json()
    response = requests.post(os.environ.get('MS_PATH'), json=json_data, headers={'Content-type': 'application/json', 'charset': 'utf-8'})
    return "The function was successfully executed"
```

- Puede encontrar el código de la función en el siguiente archivo [cloud-function.py](#)
- Adicionalmente, se debe ingresar en requirements.txt la dependencia “requests” para la instalación en el momento de ejecución de las funciones.



- Finalice presionando “Implementar” y espere a que la función se cree correctamente.
- Una vez la Cloud Function termine su proceso de construcción (aparecerá un círculo verde al lado izquierdo del nombre de la función), puede ejecutar la función al ingresar a la pestaña *Activador* y hacer clic sobre la URL de ejecución. Antes de ejecutar la función, en la base de datos del microservicio de variables debe existir una variable con id 1. Puede crearla usando Postman y generando una petición POST al endpoint `/variablecreate/` de la máquina de variables. Revise los archivos `urls.py` y `views.py` de [variables](#) para conocer cómo se envía la petición de creación. Consultando el endpoint `/variables/` puede confirmar la creación y verificar los IDs.



- Al ejecutar la función debe obtener la siguiente respuesta:

← → ↻ 🔒 us-central1-talleres-isis2503.cloudfunctions.net/api-consumption
The function was successfully executed

- Al ejecutar la ruta /measurements del microservicio o de la ruta de kong deberá ver 20 nuevas measurements guardadas en la base de datos. Cada vez que ejecute la función se guardarán 20 nuevos datos de temperatura.

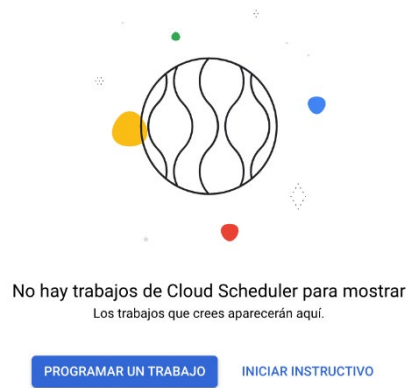
← → ↻ ⚠ No es seguro | 130.211.224.168:3000/measurements/

```
[
  {
    "id": 1,
    "variable": 1,
    "value": 25,
    "unit": "C",
    "place": "HL",
    "dateTime": "2022-11-30T05:29:24.344Z"
  },
  {
    "id": 2,
    "variable": 1,
    "value": 19.63,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 3,
    "variable": 1,
    "value": 32.71,
    "unit": "C",
    "place": "RGD-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 4,
    "variable": 1,
    "value": 40.11,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 5,
    "variable": 1,
    "value": 22.33,
    "unit": "C",
    "place": "RGD-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 6,
    "variable": 1,
    "value": 14.33,
    "unit": "C",
    "place": "HL-405",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 7,
    "variable": 1,
    "value": 42.28,
    "unit": "C",
    "place": "HL-405",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 8,
    "variable": 1,
    "value": 14.33,
    "unit": "C",
    "place": "RGD-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 9,
    "variable": 1,
    "value": 19.63,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 10,
    "variable": 1,
    "value": 25,
    "unit": "C",
    "place": "HL",
    "dateTime": "2022-11-30T05:29:24.344Z"
  },
  {
    "id": 11,
    "variable": 1,
    "value": 19.63,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 12,
    "variable": 1,
    "value": 32.71,
    "unit": "C",
    "place": "RGD-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 13,
    "variable": 1,
    "value": 40.11,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 14,
    "variable": 1,
    "value": 22.33,
    "unit": "C",
    "place": "RGD-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 15,
    "variable": 1,
    "value": 14.33,
    "unit": "C",
    "place": "HL-405",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 16,
    "variable": 1,
    "value": 42.28,
    "unit": "C",
    "place": "HL-405",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 17,
    "variable": 1,
    "value": 14.33,
    "unit": "C",
    "place": "RGD-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 18,
    "variable": 1,
    "value": 19.63,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  },
  {
    "id": 19,
    "variable": 1,
    "value": 25,
    "unit": "C",
    "place": "HL",
    "dateTime": "2022-11-30T05:29:24.344Z"
  },
  {
    "id": 20,
    "variable": 1,
    "value": 19.63,
    "unit": "C",
    "place": "C-101",
    "dateTime": "2022-11-30T05:36:28.879Z"
  }
]
```

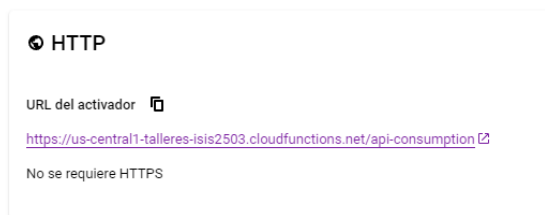
2.6. Crear Scheduler para ejecución periódica

Por si sola, la Cloud Function que creamos no se ejecuta por lo que debemos crear un Scheduler que permite programar tareas automáticas. Usaremos el Scheduler para disparar la ejecución de la Cloud Function api-consumption cada 5 minutos.

- Busque el servicio *Cloud Scheduler* en el panel de búsqueda superior o en el menú lateral izquierdo en el submenú Sin Servidores. Al ingresar le pedirá habilitar varias APIs de GCP, después de habilitar las APIs presione “Programar un Trabajo”



- Complete el formulario de creación con la siguiente información:
 - **Define el programa:**
 - **Nombre:** api-consumption-scheduler
 - **Frecuencia:** */5 * * * *
 - **Zona horaria:** Zona de ecuador (ECT)
 - **Configuración de ejecución:**
 - Tipo de destino: HTTP (Note que se ofrecen diferentes mecanismos de llamado de la función serverless, entre ellos está HTTP que es una comunicación “Remote Procedure Invocation” y otro es Pub/Sub que es “Messaging”)
 - URL: Copia la URL dada por la función para su ejecución. Esta URL se puede obtener yendo a la función “api-consumption”, pestaña “DETALLES”, clic en REST Equivalente y copie la URL (ver captura)



Las opciones de configuración anteriores se pueden ver a continuación:

Define el programa

Nombre *
api-consumption-scheduler
Debe ser único en los trabajos de la misma región.

Región *
us-central1 (Iowa)

Descripción

Frecuencia *
*/5 * * * *
Las programaciones se especifican con el formato cron de Unix. P. ej., cada minuto: * * * * *, cada 3 horas: */3 * * * *, todos los lunes a las 9:00 a.m.: "0 9 * * 1". [Más información](#)

Minuto:
Cada 5 minutos

Zona horaria *
hora de Ecuador (ECT)

Los trabajos establecidos en zonas horarias afectadas por el horario de verano pueden ejecutarse fuera de la cadencia durante el cambio a DST. Si usas una zona horaria UTC, puedes evitar el problema. [Más información](#)

CONTINUAR

Configura la ejecución

Tipo de destino *
HTTP

URL *
https://us-central1-talleres-isis2503.cloudfunctions.net/api-consumption

Método HTTP
GET

Encabezados HTTP
Cloud Scheduler configuró algunos encabezados como valores predeterminados o los quitó. [Learn more](#)

+ AGREGAR UN ENCABEZADO

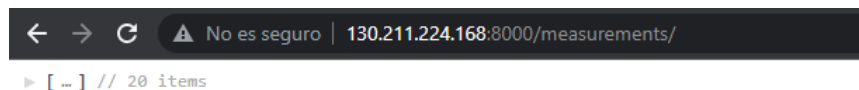
Encabezado de la autenticación
Ninguno

CONTINUAR

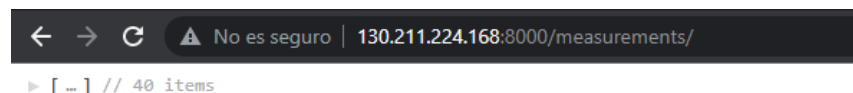
- Presionamos Crear y esperamos a que se ejecute por primera vez. La primera vez que se ejecute apreciará el estado de la última ejecución del Scheduler, como se muestra a continuación:

Cloud Scheduler	Trabajos	CREAR TRABAJO	ACTUALIZAR	EDITAR	COPIAR	PAUSAR	REANUDAR	BORRAR
TRABAJO DE SCHEDULER								
Filtro Filtrar trabajos								
☐	Nombre ↑	Estado de la última ejecución	Región	Estado	Descripción	Frecuencia	Destino	
☐	api-consumption-scheduler	Sin errores	us-central1	Habilitado		* / 5 * * * * (America/Quayaquil)	URL: https://us-central1-talleres-isis2503.cloudfunctions.net/api-consumption	

- El Cloud Scheduler que acaba de crear se ejecuta cada 5 minutos, de esta forma cada 5 minutos se actualiza la base de datos de measurements. En la siguiente imagen puede apreciar cómo tras la primera ejecución de la función de manera manual (punto 6.4 del taller), se crearon 20 measurements en la base de datos *(en mi caso tengo una extensión para el navegador que me permite ver JSONs formateados y en este caso contar los items de un arreglo)*



- Posterior a la ejecución del Cloud Scheduler, se agregaron más measurements dejando un total de 40 datos.

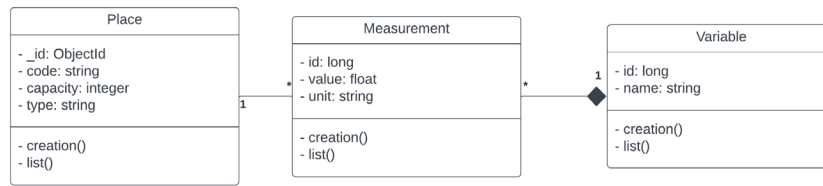


3. Entregables

Realice los siguientes puntos y **cronometre el tiempo** que le toma desarrollarlos:

- Para evolucionar el sistema, se ha decidido que se necesita guardar más información en los lugares donde se realizan las mediciones. Para esto, el equipo de tecnología de la universidad le entregó el microservicio Places. Usted debe desplegar este microservicio e integrarlo con el sistema de monitoreo que tiene desplegado actualmente. Para más información sobre el microservicio Places, se le recomienda que revise el material complementario [4.1 Microservicio Places](#).

Luego de integrar el microservicio Places, el modelo de datos resultante luce como el siguiente.



Note que en el modelo de datos incluye las clases iniciales (Measurement y Variable). En la nueva versión del modelo, el lugar incluye más atributos que se encuentran en la clase Places y la referencia desde Measurement ocurre a través del código del lugar.

Para la integración entre el sistema existente y el nuevo microservicio:

- Debe desplegar el nuevo microservicio.
 - Debe agregar el nuevo microservicio a los upstreams y servicios de Kong.
 - Debe modificar el microservicio Measurements para que cada vez que se intente crear una medida se valide que el lugar exista, cuidando así la integridad referencial de los datos distribuidos entre las DBs. Para esto debe consultar los places (llamando al microservicio Places) y validar que exista el lugar antes de crear la medida en la base de datos de Measurements.
- Cree otra Cloud Function que se ejecute cada 10 minutos y consuma los datos presentes en un Json de datos de oxígeno y los guarde en base de datos. El enlace para consumir los datos de oxígeno es <https://raw.githubusercontent.com/ISIS2503/ISIS2503-Microservices-AppDjango/master/data/oxigeno.json>

(Nota: Deben tener en cuenta que los ids de variables y los códigos de lugares que retorna el Json deben estar previamente creados en los microservicios de Variables y Places).

Como entregable de la actividad debe subir un documento PDF que tenga los siguientes elementos:

- Para cada punto de los siguientes agregue capturas y explicaciones que demuestren que lo que hizo funciona.
 - Despliegue del microservicio Places.
 - Modificación de configuración de Kong.
 - Modificación de microservicio Measurements para validar que los lugares existan.
 - Creación de algunos Places.
 - Creación de Cloud Function y scheduler para ingesta de datos de oxígeno.
 - Funcionamiento de cloud function + 3 microservicios.

- Diagrama de despliegue del sistema luego de integrar el microservicio places y la nueva función *serverless*.
- Registre el tiempo que le tomó el desarrollo de los puntos anteriores e indique si las tácticas implementadas permiten el cumplimiento del ASR 1 involucrado en el experimento.
 - En caso afirmativo, explique cómo se beneficia el ASR. De lo contrario, explique qué modificaciones podría hacer a la arquitectura (estilos o tácticas) para cumplir con el ASR.

Revise la rúbrica para el entregable del laboratorio. Esta rúbrica se encuentra vinculada a la actividad correspondiente en Bloque Neón.

Nota: Al finalizar todo el laboratorio con entregables, recuerde eliminar el despliegue (GCP Deployment Manager) o detener todas las instancias que ha utilizado en este taller, si las deja en ejecución, podría consumir la totalidad de sus créditos de GCP. Tenga en cuenta que en este laboratorio se realizan dos despliegues incluyendo el principal (variables y medidas) y el despliegue de lugares.

Comando para eliminar un despliegue (ejecutado en Cloud Shell que es diferente a SSH de las instancias de MVs):

```
>>> gcloud deployment-manager deployments delete <nombre-del-despliegue>
```

4. Material complementario

4.1. Microservicio Places

Descripción:

El microservicio Places se encarga del manejo de los lugares del campus de la universidad. En la versión actual contempla las funcionalidades CRUD para los lugares. Los datos de un lugar son código, capacidad y tipo (aula, auditorio, laboratorio y oficina). La forma en que otros microservicios referencian un lugar es a través del código del lugar. Por ejemplo, desde una medida se puede referenciar al ML515, SD402, O101 o GA405.

Implementación:

Este microservicio se desarrolló con Python 3.12, el framework web FastAPI y hace uso de una base documental MongoDB. La estructura de los paquetes es similar a los proyectos que se han trabajado en Django. El microservicio Places contiene *views*, *logic* y *models*. El tipo de la base de datos varía con respecto al resto del

sistema de monitoreo y aun así la información entre los microservicios se puede relacionar. Puede analizar el código del proyecto dentro del repositorio del laboratorio, específicamente en el siguiente enlace:

<https://github.com/ISIS2503/ISIS2503-Microservices-AppDjango/tree/master/places>

Para la integración con el resto del sistema, se le ha proporcionado un script IaC para el despliegue de este microservicio. En este script se crean 3 recursos: el firewall para el tráfico de MongoDB, la base de datos MongoDB como contenedor en una VM y la máquina virtual para el microservicio Places con las dependencias instaladas. Consulte las instrucciones del despliegue de infraestructura de los laboratorios que usan IaC para guiarse y usar este script. Para desplegar esta infraestructura desde CloudShell con deployment manager, se le recomienda que utilice un nombre diferente al del despliegue inicial. El script se encuentra en la raíz del repositorio de este laboratorio. Siga el siguiente enlace para visualizar el script.

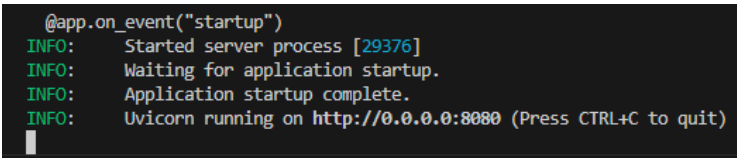
https://github.com/ISIS2503/ISIS2503-Microservices-AppDjango/blob/master/places_deployment.yaml

Para ejecutar el microservicio, ingrese a la consola SSH de la VM del microservicio (msd-places-ms) y ejecute los siguientes comandos:

```
>>> cd /home/labs/ISIS2503-Microservices-AppDjango/places
```

```
>>> sudo python3.12 main.py
```

Debería mostrarse el siguiente mensaje que indica que el servicio ya está ejecutándose.



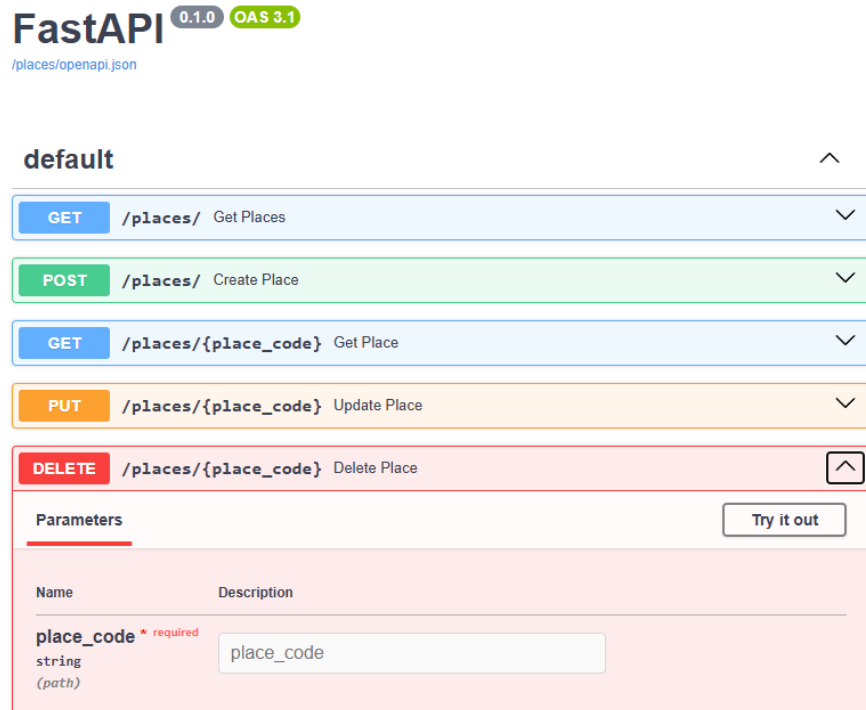
```
@app.on_event("startup")
INFO: Started server process [29376]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Documentación e interacción:

El desarrollo con FastAPI permite la documentación automática del API del microservicio a través de una interfaz gráfica con Swagger UI. Esta documentación la puede consultar navegando desde su computador personal a la siguiente URL. Reemplace la sección resaltada por la IP pública de la VM del servidor de Places.

`http://<ip_pública_ms_places>:8080/places/docs`

Se encontrará la siguiente página donde se muestran los endpoints disponibles en el servicio y documentación sobre el uso de ellos. Tenga en cuenta que puede utilizar los endpoints desde esta página utilizando el botón "*Try it out*" en cada sección.



The screenshot shows the FastAPI Swagger UI for the `/places/` endpoints. It includes the FastAPI logo (0.1.0, OAS 3.1) and the URL `/places/openapi.json`. The interface lists five endpoints:

- GET** `/places/` Get Places
- POST** `/places/` Create Place
- GET** `/places/{place_code}` Get Place
- PUT** `/places/{place_code}` Update Place
- DELETE** `/places/{place_code}` Delete Place

The **DELETE** endpoint is selected, showing its parameters:

Name	Description
place_code * required	
string (path)	place_code

A "Try it out" button is visible next to the parameters.

Se le recomienda interactuar con esta página y probar su funcionamiento. Por ejemplo, puede crear un lugar, verlo en la lista de todos los lugares, consultar el lugar específico, actualizarlo e incluso borrarlo. Así comprueba de manera manual el funcionamiento del servicio.

4.2. Otros recursos

- Además de configurar el service registry de Kong por medio del archivo declarativo (DB-less), es posible utilizar un API de administración de Kong que representa los registros en una base de datos. Para utilizar este modo puede consultar la [documentación oficial](#).
- La realización de esta guía es opcional y no es calificable pero sumamente recomendable para la implementación del patrón CQRS en microservicios. En el siguiente laboratorio les explicamos cómo

implementar en la aplicación monitoring-app el patrón CQRS, dividido en dos guías [\(Pre\) Laboratorio](#) y [Laboratorio](#).